Completion-aware search with partial witness repair for dense fixed-shape facility layout

Abstract

Dense fixed-shape facility layout requires search decisions that preserve room for the remaining facilities. We study how a verified feasible layout can guide subsequent construction under a fixed time budget. Our completion-aware search maintains a compatible completion witness and repairs only the witness facilities displaced by each proposed placement. Unaffected rectangles are reused, and an independent verifier checks every complete candidate before it can update the best layout. The main study compares partial repair with full completion regeneration and a validation-selected adaptive large neighborhood search on 240 instances with 32 to 256 facilities, two geometry families, and fill ratios 0.90 and 0.95. The common initializer succeeds on 236 instances, yielding 2,124 runs with three seeds per method. At fill 0.95, partial repair exceeds the adaptive repair control by 3.45 to 5.91 percentage points in initial-objective improvement at 10 seconds and 1.68 to 4.12 points at 60 seconds. With 256 facilities at this fill, partial repair improves the initial objective by 7.66% at 10 seconds, compared with 4.76% for full regeneration. At 60 seconds, full regeneration leads with 10.38%, compared with 8.58% for partial repair. A separate one-second comparison isolates the contribution of repair beyond unchanged witness reuse. These results support partial witness repair for early improvement of dense layouts and show how its benefit depends on problem size, packing density, and the available time.

Keywords: Facility layout, Completion repair, Fortified rollout, Large neighborhood search, Feasibility verification, Anytime search

1. Introduction

Facility layout planning considers the arrangement of functional units inside a bounded plate under hard geometric constraints and soft adjacency

preferences. The design space grows exponentially with the number of units. Mathematical programming and metaheuristics have therefore been widely used (Drira et al., 2007; Gomes de Alvarenga et al., 2000; Pérez-Gosende et al., 2021; Şahin et al., 2020; Singh & Sharma, 2006). The difficulty is stronger near full packing. A legal placement can block every completion of the remaining facilities. A one-step overlap mask does not detect this event. A completion heuristic can, when successful, provide an explicit feasible continuation. Classical constrained rollout already explains how a feasible continuation can be retained as a safeguard (Bertsekas, 2005, 2020). This invariant provides the starting point for our layout-specific repair mechanism.

Short computation matters when several candidate layouts must be compared or a feasible proposal is needed before further design decisions can be made. In that setting, improving a usable layout early has value even if a longer search can later match it. We study this computational requirement through quality at fixed cutoffs, rather than assuming a particular deployment deadline.

The complete feasible layout available after initialization provides a common starting point for comparing search strategies. A simple control can retain this layout, run any search, verify complete candidates at the end, and return the best verified layout. This gives the same feasible-return property and preserves or improves the initial objective. The computational question is therefore

> Starting from the same verified feasible layout and using the same total time, when does maintaining a compatible completion produce a better layout than incumbent-preserving search with final verification?

We answer this question with completion-aware search. Its main operation is partial witness repair. If a proposed placement intersects some rectangles in the stored completion, only those rectangles are removed and repacked. All other witness rectangles remain fixed. This is a small layout-specific destroy and repair neighborhood. A failed repair rejects only that proposal; it is not taken as evidence that the partial layout is infeasible. The current witness supplies a safe next placement, while a separate best complete layout prevents output regression. Every complete incumbent is checked by an independent verifier.

The main comparison uses 240 held out instances with 32 to 256 facilities, two geometry families, and fill ratios 0.90 and 0.95. Partial repair (M1),

full completion regeneration (M0), and a validation-selected adaptive large neighborhood search (ALNS) share the same best-contact initial packing, objective, time accounting, and final verifier. Each search runs for 60 seconds, with primary comparisons at 10 and 60 seconds. At fill 0.95, M1 exceeds the ALNS control at both cutoffs for every size. With 256 facilities, M1 also exceeds M0 at 10 seconds at both fills, but M0 leads at 60 seconds. With 128 facilities, M0 leads at 60 seconds as well. These comparisons identify both the early benefit of partial repair and the value of a broader reconstruction neighborhood when more time is available.

A separate one-second study uses final-verification construction, spatial destroy and repair, fortified rollout, and reuse without repair to isolate the mechanism. It shows that retaining a feasible layout alone does not explain M1's improvement: successful nonempty repairs provide most of the gain.

The central comparison separates two ways to spend a limited search budget: reconstructing the remaining layout or retaining compatible rectangles and repairing a smaller set. Final-verification and reuse-only controls separate this effect from initialization and from simply retaining a feasible layout.

Contributions.

1. **A partial repair mechanism.** We define the facilities displaced by a proposal and reuse every unaffected witness rectangle. A verified repair supplies a new compatible completion. The best full layout and the current compatible witness are kept as different objects.
2. **An evaluation of the quality-time tradeoff.** M1, full regeneration, and an adaptive repair control start from the same verified packing on instances with up to 256 facilities and run for 60 seconds. A separate eight-method study isolates reuse and repair at one second. Quality is paired by instance after averaging stochastic seeds, and initialization coverage is reported separately. The comparison identifies when early repair is useful and when full regeneration produces a better layout.
3. **Verification and correctness checks.** We state the feasible continuation invariant as an attributed specialization of fortified rollout, use an independent final verifier, and test it on adversarial layouts. For the small CP-SAT study, we distinguish optimality of the scaled integer model from the real-valued objective and transfer solver bounds with an explicit rounding allowance.

2. Related work

Facility layout. Fixed shape unequal area layout has been studied using mathematical programming and metaheuristics, including simulated annealing (Kirkpatrick et al., 1983), tabu search (Glover, 1989), genetic and memetic algorithms, and constructive insertion rules (Drira et al., 2007; Gomes de Alvarenga et al., 2000; Singh & Sharma, 2006). Reviews (Pérez-Gosende et al., 2021; Şahin et al., 2020) organize the field by objective and by whether departments have fixed or flexible shapes. We consider the fixed shape setting on a discrete plate. When equal-sized facilities are assigned to prescribed locations, the related assignment model is the quadratic assignment problem (Koopmans & Beckmann, 1957). Its library instances with proven optima (Burkard et al., 1997) provide an external evaluation in Section 7.5; robust tabu search (Taillard, 1991) is its reference heuristic and we run it there in its literature configuration.

Fixed outline methods. Related fixed outline floorplanning methods avoid searching absolute coordinates directly. Sequence pairs (Murata et al., 1996) and B*-trees (Chang et al., 2000) encode relative block order and induce compact packings; fixed outline algorithms then search these encodings under an outline constraint (Adya & Markov, 2003). These methods provide context for the representation limits discussed in Section 7.5. The transfer study evaluates lattice constructors on converted instances; it does not implement sequence-pair or B*-tree search.

Learning for combinatorial optimization. Neural constructive heuristics for routing and related problems (Bello et al., 2017; Kool et al., 2019), improvement policies (Wu et al., 2021) and surveys (Bengio et al., 2021; Mazyavkina et al., 2021) show that a learned scorer can replace a hand designed one inside a classical construction. A distinct line embeds learning *inside* an exact or complete solver, with learning to branch (Gasse et al., 2019) as a representative example. Our heuristic decoder uses a similar separation of roles: a classical procedure enforces feasibility, and a learned or hand designed rule can rank certified actions.

Constraint handling. The completion invariant has direct classical antecedents. Constrained rollout retains a feasible base trajectory, and fortified rollout uses its continuation when another acceptable trajectory is unavailable (Bertsekas, 2005, 2020). Under its acceptance rule, the final cost is no

worse than the base cost. Our stored witness plays the same safeguarding role. We extend this construction with marginal or learned candidate ranking and geometric repair of the compatible continuation. Section 5 states the inherited invariant and specifies how the layout implementation maintains it.

Rejecting an assignment because a future variable would lose its domain is forward checking (Haralick & Elliott, 1980). A bounded frontier gives beam search (Ow & Morton, 1988), and ordering insertions by the gap between their best options gives regret insertion (Potvin & Rousseau, 1993). More broadly, destroy and repair is the basis of large neighborhood search and adaptive large neighborhood search (Ropke & Pisinger, 2006). Partial witness repair belongs to this family. Its specific restriction is that the destroyed set is induced by overlap with a proposed rectangle, while every compatible witness rectangle is fixed and reused. We compare it with spatial destroy and repair and an adaptive control combining three removal rules with beam and regret insertion. This separates the overlap-induced neighborhood from a broader search over repair operators.

RL for layout. Learned layout agents differ in what an action is: Kakooee and Dillenburger (2024) draw walls sequentially, Ikeda et al. (2022) deploy units constructively, Luo et al. (2025) place rooms with cooperating agents, Heinbach et al. (2023) relocate facilities on a grid, and Mirhoseini et al. (2021) place macros on a chip canvas. What these systems illustrate is the range of choices for actions and geometric constraints. Our secondary single-facility translation MDP is closest to Heinbach et al. (2023); its reward and feasibility diagnostics appear in the Online Supplement, S7 and S11. The proposed M1 method uses exact marginal ranking and requires no policy training.

3. Problem

3.1. Instance and feasible set

An instance is $\mathcal{I} = (W, H, n, \{w_i, h_i\}_{i=1}^n, A, b, \theta)$: a $W \times H$ integer plate, n axis aligned facilities of fixed integer size $w_i \times h_i$, a symmetric preference matrix $A \in \mathbb{R}^{n \times n}$, boundary preferences b, and objective constants θ. A layout assigns each facility a lower left lattice position $p_i \in \mathbb{Z}^2$; write $R_i(p_i)$ for the rectangle it occupies. The feasible set is

$$\mathcal{F} = \left\{ (p_i)_{i=1}^n : \; R_i(p_i) \subseteq [0, W] \times [0, H], \; \operatorname{int} R_i \cap \operatorname{int} R_j = \emptyset \; (i \neq j) \right\}. \quad (1)$$

3.2. Objective

On $\mathcal{F}$ the reported objective is

$$J(g) \equiv J_\beta(g) \;=\; \beta_{\max} + w_{\text{adj}} \sum_{i<j} c_{ij}(g) \;+\; \sum_i b_i(g), \tag{2}$$

where c_{ij} rewards proximity between facilities that prefer it ($A_{ij} < 0$), rewards a normalized separation between those that prefer distance ($A_{ij} \geq 0$), and b_i pays a boundary bonus when facility i is flush against a wall it prefers. Distances are center to center with each rectangle's directional radius removed, and are normalized per pair by the largest separation the plate permits, so c_{ij} is scale free. The full expressions are reproduced in Appendix A, and the objective evaluator is included in the reproducibility archive.

The constant $\beta_{\max}$ is the no-overlap bonus used by the evaluator. It does not change rankings or raw paired differences within an instance, but it is included in every reported value and in the denominator of each normalized improvement. Outside $\mathcal{F}$ two further terms appear, and they matter only because the secondary penalty-based improvement formulation operates there: an overlapping pair contributes zero adjacency, an overlapping facility forfeits its whole boundary bonus, total overlap area is penalized, and the constant $\beta_{\max}$ is replaced by a graded overlap bonus. Our analysis treats Eq. (2) on $\mathcal{F}$ as the optimization problem and the off-feasible penalty and graded bonus as components of the penalty-based improvement MDP.

3.3. Common objective evaluation

Every method in the scenario and generated-suite experiments uses the same vectorized evaluator of Eq. (2). Its numerical agreement with the reference scenario evaluator is checked on two manually defined 32-facility scenarios: OFFICE (36×24, fill ratio 0.54) and CLINIC (28×16, fill 0.68). The checks cover random, lattice aligned, heavily overlapping, and wall-flush layouts under both affinity sign conventions. The absolute difference is 0.0 in all checked cases.

4. Why local legality is insufficient

For construction, let $\mathcal{P}_t$ be the facilities already placed. Facility $i \notin \mathcal{P}_t$ can be placed at

$$\mathcal{L}_i(s_t) = \{p \in \mathbb{Z}^2 : 0 \leq p_x \leq W - w_i,\ 0 \leq p_y \leq H - h_i,\ S_i(p) = 0\}, \tag{3}$$

where $S_i(p)$ is the occupied area under its proposed rectangle. An integral image tests every lattice position. The following properties describe local legality and the exact objective increment along a completed construction.

Proposition 1 (Local feasibility and exact marginal)**.** *If every step uses Eq.* (3) *and every facility is placed exactly once, the complete layout is contained in the plate and overlap free. Moreover, the objective increment from placing i is exactly the sum of pair terms between i and $\mathcal{P}_t$ plus i's boundary term.*

Proof. Containment is explicit. Induction over placements gives nonoverlap. Since a legal placement creates no overlap and no placed facility moves, no existing pair or boundary term changes; only terms involving i are added. □

This proposition is conditional on completing all n placements. It gives no continuation guarantee. For example, on a 5×2 plate, placing a 1×1 rectangle at $(1, 0)$ leaves each of two 2×2 rectangles individually placeable, but they cannot both be placed. The original three-rectangle instance is feasible if the small rectangle occupies the unit gap between the two large ones. Deciding whether a partial state has a completion contains rectangle packing feasibility and is NP-hard in general (Fowler et al., 1981). This is the gap addressed next; further penalty and masking diagnostics are reported in the reproducibility record and Online Supplement.

5. Witness certified construction

5.1. A sufficient certificate

Let $\mathcal{V} = \{s : \text{some sequence of legal placements completes } s\}$ be the set of viable partial layouts. We bracket $\mathcal{V}$ rather than decide it. Let $\mathrm{FF}(s)$ denote the outcome of first fit packing in largest first order. We process the unplaced facilities in decreasing area and place each at the first position of $\mathcal{L}_i$ in row major order. Define the predicate

$$\mathcal{C}(s) \;=\; \Big[\,\mathrm{FF}(s) \text{ places every unplaced facility}\,\Big]. \tag{4}$$

$\mathcal{C}$ is sufficient but not necessary for viability: if it holds it exhibits an explicit completion, so $s \in \mathcal{V}$; if it fails, s may still be viable. It costs $O(nWH)$, the same order as the mask itself.

A certificate is only useful here if it returns the completion it found. We therefore take $\mathcal{C}$ to return a *witness*, the explicit list of placements made by

the completion heuristic. A verifier checks that all facilities occur exactly once with their fixed dimensions, every coordinate is within the plate, rectangles have disjoint interiors, and the completion agrees with the committed prefix. It does not call the objective or the witness generator. Once verified, the remaining entries stay a valid completion after any one of their placements is committed.

This is the feasible-continuation invariant used by constrained and fortified rollout (Bertsekas, 2005, 2020). The following layout specialization gives the conditions that the decoder and repair procedure must maintain.

Theorem 1 (Feasible-continuation invariant)**.** *Consider a decoder that (i) obtains and verifies a completion witness w_0 for s_0, abstaining if none is available; (ii) commits a proposed placement only when its successor comes with a newly verified completion witness; and (iii) if no proposal is accepted, commits the selected facility at its position in the current witness and carries the remainder of that witness forward. Then the decoder, conditional on successful initialization and before a time cutoff, has an available commitment at every step; every completed layout it returns lies in $\mathcal{F}$.*

Proof. By induction on t with hypothesis "s_t has a witness w_t". The base case is (i). Given w_t, condition (ii) either commits a proposal and supplies a verified w_{t+1}, or condition (iii) commits the selected facility at the position recorded in w_t. In the latter case that rectangle is legal in s_t and the remaining entries $w_t \setminus \{i\}$ complete the successor, so they form w_{t+1}. Thus a witness exists after every commitment and a commitment is always available. Every committed rectangle is legal, so Proposition 1 applies at every step and the completed layout lies in $\mathcal{F}$. □

The guarantee is conditional on initialization. The decoder may abstain on a feasible instance if the initial packer fails, but subsequent candidate ranking does not reduce return coverage. This applies to any selection rule that retains the witness fallback. A heuristic that instead commits an uncertified action after all tested candidates fail is only certificate guided; its reliability is empirical. The Online Supplement distinguishes those diagnostic decoders.

In every timed method we also retain a separate best verified complete layout. The compatible witness supports the next commitment; the best complete layout is the output incumbent and need not agree with the current prefix. If time expires during an incomplete completion or repair, the method

returns this incumbent. Thus every initialized method has the same feasible-return coverage and cannot return a value below its common initial layout.

5.2. Partial repair of the compatible completion

Let P be the committed prefix, U the unplaced facilities, and w a verified placement of U that completes P. Consider placing $i \in U$ at p. After checking containment and legality against P, define the displaced set

$$D(i, p; w) = \{j \in U \setminus \{i\} : \ \operatorname{int} R_j(w_j) \cap \operatorname{int} R_i(p) \neq \emptyset\}. \tag{5}$$

We remove i from its position in w, keep every facility in $U \setminus (D \cup \{i\})$ fixed, and try to repack only D. The committed prefix, the proposed rectangle, and the retained witness rectangles are obstacles. In the evaluated method, a first-fit repair is attempted when $|D| \leq 4$; a failed attempt rejects the proposal but says nothing about global feasibility.

Lemma 1 (Verified witness reuse)**.** *If the repair places every facility in D inside the plate without overlap with the fixed obstacles or with another repaired facility, then the retained and repaired rectangles form a completion after committing (i, p). If $D = \emptyset$, the retained witness alone forms this completion.*

Proof. The retained rectangles are pairwise disjoint and avoid P in w. By definition of D, they also avoid $R_i(p)$. The repaired rectangles avoid all fixed rectangles and one another. Every facility in $U \setminus \{i\}$ therefore occurs exactly once in a rectangle contained in the plate. □

The lemma establishes correctness of the witness update. By restricting repair to the displaced set, the method packs $|D|$ facilities while reusing the placements of all remaining facilities. The final verifier still checks the whole completion, so the reduction applies to witness generation while retaining the full check. The experiments log $|D|$, repair success, reuse, full completion calls and verifier calls to measure the resulting quality-time tradeoff.

5.3. Completion-aware search with partial repair

Algorithm 1 states M1, the principal method. Facilities are visited in decreasing area in the first reconstruction and in randomized orders afterwards. Positions are ranked by their exact marginal contribution. The position already stored for the selected facility is appended to the top κ candidates, so

a compatible commitment is always available. The search continues until its post-initialization deadline.

Algorithm 1 Completion-aware search with partial witness repair (M1).

$(ok, w) \leftarrow$ BestContact($\emptyset$); verify w
if $\neg ok$ **then return** Not Initialized
$g_{\text{best}} \leftarrow w$; $J_{\text{best}} \leftarrow J(w)$
$B \leftarrow$ Clock() $+ T$ ▷ post-initialization deadline
while Clock() $< B$ **do**
    $P \leftarrow \emptyset$ ▷ committed prefix for one reconstruction
    for facility i in the selected order **do**
        $Q \leftarrow$ top κ legal positions by exact marginal score
        append i's position in w to Q if absent
        for $p \in Q$ in decreasing score **do**
            $D \leftarrow D(i, p; w)$
            if $D = \emptyset$ **then**
                $w' \leftarrow w \setminus \{i\}$ ▷ reuse unchanged remainder
            else if $|D| \leq 4$ **then**
                $w' \leftarrow$ Repair($D \mid P, (i, p), w \setminus (D \cup \{i\})$)
            else
                $w' \leftarrow$ Failure
            end if
            if $w' \neq$ Failure **then**
                $g' \leftarrow P \cup \{(i, p)\} \cup w'$
                $(ok, J', t_{\text{done}}) \leftarrow$ VerifyAndScore(g')
                if $t_{\text{done}} > B$ **then**
                    discard g'; **return** g_{best}
                end if
                if ok **then**
                    record g' as an eligible completed layout
                    if $J' > J_{\text{best}}$ **then**
                        $g_{\text{best}} \leftarrow g'$; $J_{\text{best}} \leftarrow J'$
                    end if
                    commit (i, p) to P; $w \leftarrow w'$; **break**
                end if
            end if
        end for
        if the deadline is reached **then**
            return g_{best}
        end if
    end for
    $w \leftarrow g_{\text{best}}$ ▷ start the next reconstruction from the incumbent
end while
return g_{best}

The witness position in Q has $D = \emptyset$, so the inner loop always has a verified fallback unless the deadline interrupts it. In that case the separate complete incumbent is returned. M0 replaces the repair branch by full regeneration of all remaining facilities; like M1, it reuses the carried witness action itself. R0 accepts only nonfallback proposals with $D = \emptyset$ and never calls repair, so it isolates witness reuse from repair. B3 evaluates the full completed objective for each successful full regeneration and selects a non-worsening continuation, which is fortified rollout.

With r unplaced facilities, full first-fit completion costs $O(rWH)$ per query. Partial generation costs $O(|D|WH)$ when $|D| \leq 4$, while the independent whole-completion verifier remains $O(nWH)$. To measure the net computational effect, we record calls, repair sizes and wall-clock quality. All matched-time experiments use $\kappa = 32$.

5.4. Comparison from the same initial layout

Table 1 lists the methods and controls used in the two matched-time studies. Every method starts from the same verified best-contact layout w_0, keeps it as an incumbent, and uses the same independent verifier. B1 isolates the value of intermediate completion checks. B2 is a spatial large-neighborhood control: it removes 2 to 16 facilities near a random anchor and repacks them. B2S uses the identical implementation but limits the destroy size to 2 to 4. B3 is fortified rollout. M0, R0 and M1 have the same marginal candidate order. For every nonfallback proposal, M0 regenerates all remaining facilities, R0 accepts only $D = 0$, and M1 applies Eq. (5) with a repair cap of four. The 60-second study compares M0 and M1 with ALNS. This control combines random, spatial, and low-contribution removal with beam and regret insertion, giving six operator pairs. Operator weights adapt to search outcomes, and temperature-based acceptance allows nonimproving moves while the best verified layout is retained separately. Its configuration is selected on disjoint validation instances before testing. The one-second study uses the eight methods B0 through M1 listed in the table, excluding ALNS. The guillotine and nonslicing instance families are described in Section 6.1.

The primary comparison uses a common best-contact initializer and requires no policy training. M0 and M1 rank candidates by exact marginal objective gain. Supporting full-regeneration tests separate the witness's feasibility safeguard from the selector's effect on quality: exact marginal ranking matches or exceeds the learned selectors in most $n = 64$ cells. The Online

Table 1: Methods in the matched-time studies. All start from the same initial layout and retain the best verified complete layout found.

ID	method	role
B0	initial witness	return w_0 without post-initialization search
B1	final verification	randomized construction; verify only complete candidates
B2	spatial destroy and repair	independent multi-facility improvement search
B2S	small spatial destroy and repair	B2 with destroy size restricted to 2 to 4
B3	fortified rollout	rank by completed objective and enforce non-worsening
M0	full regeneration	marginal order; regenerate nonfallback proposals
R0	reuse only	marginal order; accept nonfallback proposals only when $D = 0$
M1	partial repair	marginal order with bounded witness reuse and repair
ALNS	adaptive destroy and repair	weighted removal and insertion operators; separate current and best layouts

Supplement reports initialization coverage and selector comparisons, with complete results in the reproducibility archive.

6. Experimental design

6.1. A generated benchmark

The two reference scenarios measure solver variability on fixed briefs. To evaluate behavior across problems, we generate a held out suite and treat the *instance* as the unit of analysis.

Feasibility by construction. To ensure that each high-density instance has a feasible solution, the first family is built from a guillotine partition, recursively slicing the plate into n disjoint integer rectangles and shrinking each inside its own slot.

The second family starts from a tiled five-rectangle pinwheel with no full horizontal or vertical guillotine cut. A source rectangle is selected and split at an integer coordinate. The split is accepted only if the resulting tiling still has

no full guillotine cut. This procedure is repeated until the tiling has n source slots. The rectangles are then reduced by a common scale factor, rounded down to integer dimensions, and enlarged one cell at a time without leaving their respective slots until the target fill is reached. The nonslicing condition is checked on the complete source tiling before this reduction. After the reduction, gaps are present and the same full-cut test for a tiling is no longer applicable. Thus, the family label describes the source tiling; the reduced instances have verified feasible packings, while their nonslicing status under definitions allowing dead space remains unchecked.

For both families, the lower-left corners of the source slots give a feasible witness. Its boundary and overlap conditions are independently verified before any solver is run, but the coordinates are withheld from every method. Details of the instance generation, structural checks and execution scripts are documented in the public code repository.

Design and splits. The main 60-second study crosses $n \in \{32, 64, 128, 256\}$ with both geometry families and nominal fills 0.90 and 0.95. Each geometry, size, and fill cell has 10 attempted instances at fill 0.90 and 20 at fill 0.95, totaling 240. The maximum plate side is 44, 44, 72, and 100 for the four sizes, respectively; the sampled mean-area range remains 9 to 30. This increases plate capacity with size rather than forcing all large instances onto the smaller grid. The separate one-second study uses 200 attempted instances at fill 0.95 and 80 at fill 0.90, crossing $n \in \{32, 64\}$ with both geometries. Its instance tags and timing records are kept separate from the 60-second study. The generator also supplies small instances for objective calibration and secondary construction studies described in the Online Supplement, S12.

Plate aspect ratio, facility types, affinities and boundary preferences vary by instance. The three randomized objective constants are drawn once per suite cell. Integer dimensions make achieved fill approximate; the result files record each instance's achieved fill. Training, validation and test use separate seed ranges and generator tags. Full cell settings, including batch size, are retained because they determine the random draws and instance identity.

6.2. Comparison through 60 seconds

M0, M1, and ALNS each use three search seeds on every initialized instance. One uninterrupted run records the best eligible layout at 0, 0.1, 0.3, 1, 3, 10, 30, and 60 seconds after initialization. Thus the short and long

cutoffs measure the same search trajectory. M0 and M1 use $\kappa = 32$, one completion attempt, and M1's repair cap of four, without additional tuning.

ALNS is a layout-specific adaptation of adaptive large neighborhood search (Ropke & Pisinger, 2006). We compare four configurations with removal caps 4, 8, or 16 and beam widths 4 or 8. Validation crosses $n \in \{64, 128\}$, both geometries and both fills, with two instances per cell and two search seeds. All 16 instances initialize. The configuration with the largest seed-averaged mean improvement at 10 seconds is selected; ties are resolved by a fixed alphabetical rule. The selected configuration removes 2 to 4 facilities and uses beam width four and four ranked positions, adding an original position when legal. Its validation improvement is 7.80%, compared with 5.83% for B2 and 6.99% for B2S. Test results do not enter this selection. The full operator rules and configuration comparison are in the Online Supplement, S1.

6.3. One-second mechanism study

The dense study uses 50 attempted instances per geometry and size cell at fill 0.95; its boundary check uses 20 per cell at fill 0.90. The common best-contact initializer succeeds on 196 of 200 dense instances and 79 of 80 boundary instances. The eight methods in Table 1, excluding ALNS, run to a one-second cutoff. Values at 0.1 and 0.3 seconds are prefixes of that run. The dense study uses two seeds per stochastic method and the boundary check uses three.

B2 removes facilities near a random anchor and repacks them, sampling from the four strongest among 12 candidate positions. Only a complete verified improvement can replace its incumbent. Its destroy cap of 16 was selected from 4, 8, 12, and 16 on 20 disjoint validation instances at $n = 64$, fill 0.95, and 0.3 seconds. B2S uses the same implementation with cap four. M0 and M1 retain the settings used in the 60-second study.

6.4. Secondary methods

The principal methods are defined in Table 1. Secondary studies compare annealing, tabu search, a genetic algorithm, greedy and regret insertion, beam search, and learned constructors. Their settings, dataset coverage and statistical protocols are collected in the Online Supplement, S12; policy architecture and training costs are in S13. These diagnostics do not enter the primary paired comparison.

6.5. Cost accounting

The matched-time studies use CPU implementations and include candidate scoring, completion or repair, and independent verification in search time. A candidate is eligible only when verification and objective evaluation have both finished by the cutoff. Nonpreemptive work can return later, but its candidate is discarded. Measured initialization time is reported separately and charged equally to every method in total-time summaries.

The 60-second study uses eight processes pinned to distinct physical cores of an AMD Ryzen 7 9700X on Windows 10, with Python 3.10.18 and NumPy 2.2.5. All method and seed runs for one instance execute on the same core in a deterministic shuffled order; BLAS and OpenMP thread counts are one. Validation uses the same allocation. This measures matched-time quality under eight-worker throughput conditions, rather than dedicated-machine latency. The separate one-second study uses one CPU process on the same processor, with Python 3.7.16 and NumPy 1.21.5. Timing results from these environments are not pooled. The CP-SAT audit uses OR-Tools 9.15.

Secondary learned inference uses a GPU. Its wall time is reported separately from the CPU search comparisons, together with training cost. Full objective evaluations, candidate position scores, and network forward passes are also distinguished because one call entails different work in each method.

6.6. Statistics

Quality is conditional on the fixed common initialized set. For method m, the normalized improvement is $100[J_m(t) - J(w_0)]/J(w_0)$; the constant no-overlap bonus remains in its denominator. Differences in this improvement are reported in percentage points, together with raw objective differences in the result archive. Seeds are averaged within each instance before pairing.

The primary 60-second-study contrasts are M1 minus M0 and M1 minus ALNS at 10 and 60 seconds, separately by size and fill. The two geometry families receive equal weight. We use 10,000 paired bootstrap samples, resampling instances within geometry. The 95% intervals are descriptive and pointwise, without adjustment for the family of comparisons. Smaller cutoffs, 30-second quality, and time to 1%, 5%, and 10% improvement are supporting summaries. For target times, non-hits are right-censored at 60 seconds and contribute 60 seconds to the restricted mean; they are not dropped. Companion total-time summaries add measured initialization time.

In the one-second study, 5,000 paired instance-bootstrap samples are used. Its dense contrast is M1 minus B1 at one second; R0, B2S, and

M0 isolate reuse, repair scale, and regeneration. The fixed practical-effect threshold is one percent of the initial objective. The two studies are analyzed separately. Protocols for the secondary comparisons are in the Online Supplement, S12.

7. Results

7.1. Quality at 10 and 60 seconds

The main study initializes 236 of 240 instances. The four failures occur at $n = 32$, fill 0.95, with two in each geometry family; they remain in the coverage denominator. All attempted instances with 128 or 256 facilities initialize. The common initialized set yields 2,124 runs, comprising three methods and three seeds per instance.

Table 2 reports both primary contrasts for every size and fill. At fill 0.95, M1 exceeds the selected ALNS control at 10 and 60 seconds for all four sizes. The differences range from 3.45 to 5.91 percentage points at 10 seconds and 1.68 to 4.12 at 60 seconds; the corresponding pointwise intervals are above zero. Thus the dense-layout advantage extends beyond the one-second spatial controls to an independently selected adaptive repair method.

Table 2: M1 minus control in percentage points of initial-objective improvement. Entries give the paired mean and descriptive pointwise 95% interval, with equal weight for the two geometries. Positive values favor M1. Each fill 0.90 row has 20 initialized instances; fill 0.95 has 36 at $n = 32$ and 40 at each other size.

n	fill	10 seconds		60 seconds	
		M1−M0	M1−ALNS	M1−M0	M1−ALNS
32	0.90	−1.60 [−3.05, −0.28]	+1.01 [−0.57, +2.52]	−0.10 [−1.22, +0.95]	+0.66 [−0.64, +2.03]
64	0.90	−1.98 [−3.21, −0.78]	+1.36 [+0.35, +2.41]	−0.79 [−1.78, +0.18]	−1.74 [−2.76, −0.79]
128	0.90	−2.22 [−3.84, −1.08]	+5.04 [+3.61, +6.43]	−2.59 [−3.74, −1.68]	−0.41 [−1.66, +0.88]
256	0.90	+3.98 [+2.79, +5.25]	+10.00 [+8.98, +11.08]	−4.70 [−6.37, −3.16]	+3.93 [+2.69, +5.18]
32	0.95	+1.35 [+0.34, +2.28]	+3.45 [+2.47, +4.44]	+1.23 [+0.04, +2.31]	+2.97 [+2.09, +3.87]
64	0.95	−1.01 [−2.24, +0.19]	+4.16 [+3.39, +5.01]	+0.52 [−0.57, +1.52]	+1.68 [+0.98, +2.44]
128	0.95	−0.98 [−2.31, +0.37]	+5.10 [+4.19, +6.04]	−2.35 [−3.51, −1.20]	+3.08 [+2.33, +3.90]
256	0.95	+2.90 [+2.08, +3.77]	+5.91 [+5.35, +6.46]	−1.80 [−2.89, −0.74]	+4.12 [+3.71, +4.54]

The comparison with M0 depends on size and cutoff. At $n = 32$, fill 0.95, M1 leads by 1.35 points at 10 seconds and 1.23 at 60 seconds. At $n = 64$, both intervals include zero. At $n = 128$, the 10-second interval includes zero, while M0 leads by 2.35 points at 60 seconds. At $n = 256$, M1 leads by 2.90 points at 10 seconds (95% interval 2.08 to 3.77), but M0 leads by 1.80 points

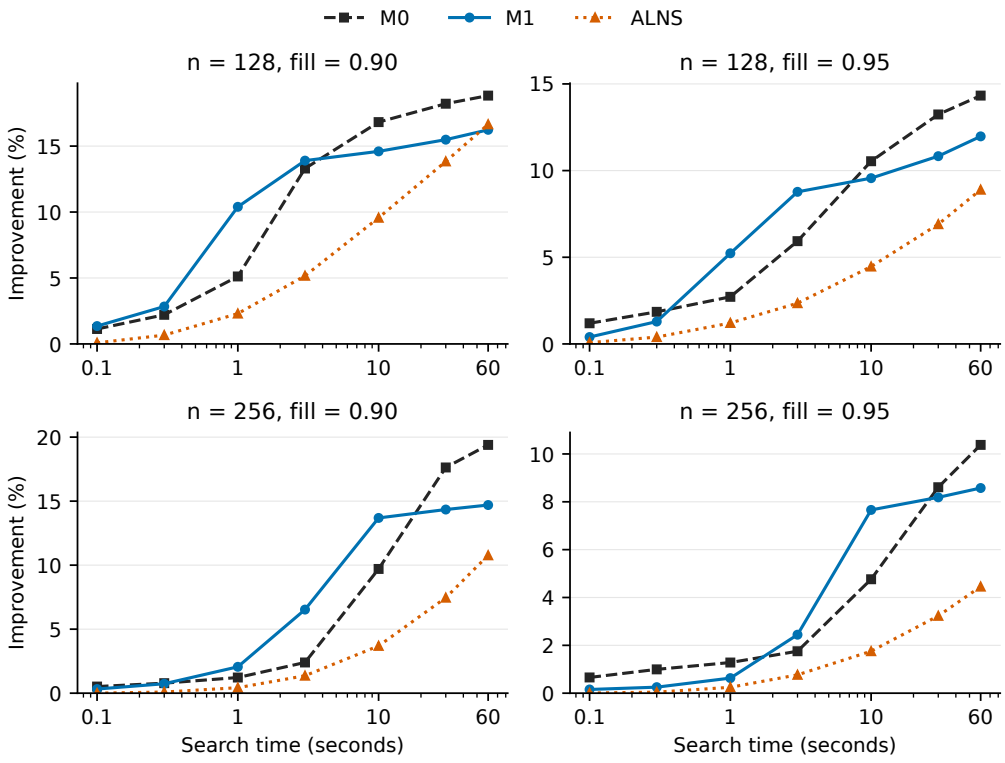

Figure 1: Quality through 60 seconds for 128 and 256 facilities. Points are mean initial-objective improvements after averaging three seeds within instance and giving the two geometry families equal weight. Each shorter cutoff is a prefix of the same 60-second run; connecting lines guide the eye. Time excludes the common initializer. Table 2 gives pointwise intervals for the primary contrasts.

at 60 seconds (0.74 to 2.89). For this last cell, M1, M0, and ALNS improve the initial objective by 7.66%, 4.76%, and 1.75% at 10 seconds, and by 8.58%, 10.38%, and 4.45% at 60 seconds.

At fill 0.90, M0 exceeds M1 at 10 seconds for $n = 32$, 64, and 128. At $n = 256$, M1 leads M0 by 3.98 points at 10 seconds, while M0 leads by 4.70 points at 60 seconds. ALNS also exceeds M1 at $n = 64$ and 60 seconds. These cells show that the benefit is not uniform across density or size, even at an early cutoff.

Figure 1 makes the tradeoff visible at the larger sizes. Partial repair gives a useful early improvement at $n = 256$, while full regeneration continues to

improve and overtakes it by 60 seconds. Time-to-target summaries provide a related check. At $n = 256$, fill 0.95, M1 reaches a 5% improvement in 94.2% of runs with a restricted mean of 9.76 seconds, compared with 90.0% and 20.73 seconds for M0. For the more demanding target, the fractions reaching a 10% improvement within 60 seconds are 56.67% for M0, 20.00% for M1, and zero for ALNS, pooling the two geometries. Their restricted mean search times are 39.15, 51.74, and 60.00 seconds. Thus M1's early quality advantage does not imply faster attainment of a larger improvement target. Geometry-specific quality and coverage, pooled target times, and complete quality curves are in the Online Supplement, S1.

7.2. What partial repair contributes

The separate one-second mechanism study provides controls for the repair operation. On its 196 initialized dense instances, M1 exceeds B1 by 5.93 percentage points (95% paired interval 4.99 to 6.88). Its gains over reuse without repair (R0), spatial repair (B2), and small spatial repair (B2S) are 6.39, 7.02, and 6.33 points, respectively. M1 improves the initial objective by 7.98% to 9.65% across the four geometry and size cells, compared with 1.72% to 4.44% for B1. Detailed budget tables are in the Online Supplement, S2.

Successful nonempty repairs account for 6,215 incumbent updates and 92% of M1's logged objective gain, compared with 1,380 updates from unchanged reuse. Mean proposed displaced-set size ranges from 1.95 to 2.72, and 2.19% to 3.91% of attempted nonempty repairs of size at most four succeed. These are descriptive mechanism counts, not independent inferential samples. Together with the R0 contrast, they show that the repair branch carries most of the improvement.

The pooled M1 minus M0 difference at one second is 0.25 points (-0.42 to 0.92). In the separate fill 0.90 boundary check, M0 leads by 1.87 points (1.17 to 2.60) on 79 initialized instances. These results complement, rather than pool with, the 60-second comparison.

7.3. Verification and timing

All 17,228 stored layouts in the 60-second study pass the independent audit, with zero objective recomputation error. The audit also checks instance identity, shared initialization, trace monotonicity, and cutoff eligibility. No method records a final verification failure. Mean process-CPU to wall-time ratios are 0.979 to 0.981, and their medians are 0.997 for all three methods. The largest function-return overrun is 66.4 ms; completed candidates after

the deadline are excluded. The one-second studies are audited separately, with 8,820 dense and 5,214 boundary budget layouts verified. Detailed timing distributions are reported in S1 and S2.

7.4. Small-instance objective calibration

To complement the paired rankings with bounds on objective quality, we freeze a dedicated small plate family from the same generator ($n = 8$, fills 0.40 to 0.75, 20 instances each, mean facility area 6 to 10 so plates stay at 66 to 195 cells). The CP-SAT model (Google OR-Tools, 2026) has integer lower-left coordinates $X_i \in [0, W - w_i]$ and $Y_i \in [0, H - h_i]$, interval variables of fixed size, and a native NoOverlap2D constraint. For every nonzero pair, an allowed-assignment table has columns (u, v, q), where $u = X_j - X_i$, $v = Y_j - Y_i$, and $q = \mathrm{round}(10^4 w_{\mathrm{adj}} c_{ij})$. Boundary coefficients are rounded at the same scale and attached to reified wall-contact variables. The no-overlap bonus is constant on $\mathcal{F}$ and is added without rounding.

Let $a_\ell(g)$ denote an unrounded active pair or wall term, let M be the number of such terms and let $s = 10^4$. For every feasible layout,

$$\widehat{J}(g) = \beta_{\max} + \frac{1}{s}\sum_{\ell=1}^{M} \mathrm{round}(s a_\ell(g)), \qquad |J(g) - \widehat{J}(g)| \le \delta := \frac{M}{2s}. \quad (6)$$

Thus a CP-SAT upper bound $\widehat{U}$ transfers to the real objective as $U = \widehat{U} + \delta$, and an optimizer of $\widehat{J}$ is within 2δ of the real optimum. CP-SAT proves 45 of 60 *quantized* models optimal within 600 seconds and returns bounds for the other 15. The audit independently checks all returned layouts and reevaluates their real objective. The largest δ is 0.00215 and the largest observed $|J - \widehat{J}|$ is 0.000322. The real-valued guarantee is the transferred upper bound with its explicit rounding allowance.

The ranking replicates the paired comparison on this small family. The bounds calibrate objective quality at $n = 8$ for the same objective. Optimality gaps for the larger layout instances and bounds for the separate QAPLIB objective require their own studies.

7.5. Secondary boundary studies

Secondary studies examine completion reliability and representation limits. Unfiltered one-step construction loses completion reliability as fill approaches 0.95, showing the limits of local legality checks. Constructor feasibility tests up to 256 facilities complement M1's quality evaluation in Section 7.1. Under the QAPLIB objective, swap improvement performs well

Table 3: Small-instance calibration at $n = 8$. The first three numeric columns are real-objective shortfalls from the returned CP-SAT solution on the 45 quantized models proved optimal. The last column averages the relative gap bounds obtained from the transferred real upper bound U on the full set of 60 instances (59 feasible returns for regret 4); it is not a worst-case bound. Objectives and bounds are positive here, so $(U - J)/U$ upper-bounds the relative gap to the unknown real optimum. Regret 4 is infeasible on one instance and is excluded rather than imputed.

method	mean	median	max	mean gap bound
beam width 32	**0.40%**	**0.19%**	2.66%	≤0.85%
SA 45 000, lattice	1.48%	0.85%	9.79%	≤1.74%
regret 4 insertion	1.94%	1.41%	6.05%	≤2.35%
tabu 45 000, greedy start	2.62%	1.96%	7.31%	≤2.88%
largest first greedy	2.77%	2.13%	7.31%	≤3.00%

when geometric feasibility constraints are absent. Tests on integer-converted MCNC/GSRC floorplans assess direct lattice construction; the conversion changes rectangle geometry and dead space, so the results cannot be compared directly with published full-resolution floorplans. Selected tables are in the Online Supplement and complete results are in the archive. These studies provide context for the primary matched-time comparison.

8. Discussion

The main comparison identifies a practical role for partial repair: obtaining an improved dense layout within a limited search budget. At fill 0.95, M1 outperforms the selected adaptive repair control at 10 and 60 seconds across all four sizes. The one-second reuse control provides complementary evidence that nonempty repairs, rather than simply retaining a feasible layout, account for most of the improvement.

Full regeneration remains a competitive alternative. With 256 facilities, partial repair gives higher quality at 10 seconds at both fills, while full regeneration leads at 60 seconds. At 128 facilities, full regeneration also leads at 60 seconds. Partial repair reduces the number of facilities passed to the completion heuristic, but it fixes every unaffected rectangle and rejects proposals that displace more than four facilities. The observed crossover is consistent with this tradeoff between cheaper repair and a smaller neighborhood. It does not establish that repair cost alone causes the difference, since ranking, verification, and the sequence of accepted placements also affect search.

These results matter when a planner compares several feasible alternatives before choosing which designs deserve more computation. Partial repair can provide an early improved candidate, while further search may favor full regeneration. This interpretation concerns computational performance on the tested instances; the experiments do not measure industrial decision times. A time-dependent combination of repair and full regeneration is a natural next question, but the present comparison does not evaluate such a hybrid.

The compatible completion and the best complete layout serve distinct roles. The former supports the next commitment, while the latter protects output quality. All compared methods retain a verified incumbent and therefore share feasible-return coverage and non-worsening output after initialization. Their quality differences measure the search operations performed with the remaining time, not a difference in whether a feasible result is returned.

The small CP-SAT and QAPLIB studies provide complementary calibration. CP-SAT bounds the same objective at $n = 8$ with an explicit integer-to-real allowance. QAPLIB shows that swap search is strong when geometric feasibility is absent. Neither supplies an optimality bound for the dense 32- to 256-facility cells.

9. Limitations

1. **The tested regime is bounded.** The main study covers 32 to 256 facilities, two generated geometry families, and budgets through 60 seconds. Each geometry and size cell has 20 attempts at fill 0.95 and 10 at fill 0.90, with three seeds. The smaller fill 0.90 sample gives less precise estimates. The pointwise intervals do not provide a simultaneous confidence statement across all sizes, fills, controls, and cutoffs.
2. **Initialization remains incomplete.** The main study initializes 236 of 240 instances, including 18 of 20 in each $n = 32$, fill 0.95 cell. The one-second dense study initializes 196 of 200. Quality comparisons are conditional on these common initialized sets; feasible instances can still defeat the initializer.
3. **Repair neighborhood coverage.** The adaptive control combines six operator pairs and uses a configuration chosen from four candidates on 16 validation instances. This is a defined layout-specific ALNS baseline, not an exhaustive evaluation of adaptive search. Other operators, CP-SAT repair, or more tuning may improve it. M1's cap of four and exact marginal candidate order are also empirical choices.

4. **The representation and objective limit external validity.** Core experiments use fixed-orientation axis-aligned rectangles on a lattice, without corridors or doors. Rotation is enabled only in the MCNC/GSRC transfer study. Integer conversion changes those benchmark geometries and prevents direct comparison with full-resolution topological fixed-outline methods. The 60-second study also increases plate capacity with facility count, so size-dependent results concern this joint scaling of n and the plate.
5. **Optimality evidence is local.** CP-SAT closes 45 of 60 scaled integer models at $n = 8$. The real-objective allowance is small but nonzero, and neither these bounds nor QAPLIB optima transfer to larger sizes or another objective.
6. **Timing depends on the execution setting.** The main study uses eight pinned single-threaded workers, not an otherwise idle single-worker machine. The one-second study uses a separate software environment. These timings are kept separate. Learned inference appears only in secondary GPU diagnostics and is not a hardware-normalized speedup over CPU search.

10. Conclusion

This paper develops completion-aware search with partial witness repair for dense fixed-shape facility layout. A proposed placement determines which witness rectangles must move, while every compatible rectangle is reused. An independent verifier checks complete candidates, and a separate best layout protects the quality of the returned solution.

The main study evaluates M1, full completion regeneration, and a validation-selected adaptive repair control on 240 attempted instances with up to 256 facilities. At fill 0.95, M1 exceeds the adaptive control by 3.45 to 5.91 percentage points in initial-objective improvement at 10 seconds and 1.68 to 4.12 points at 60 seconds. With 256 facilities at this fill, M1 gives 7.66% improvement at 10 seconds, compared with 4.76% for full regeneration. By 60 seconds, full regeneration gives 10.38%, compared with 8.58% for M1. The separate one-second study attributes most of M1's gain to nonempty repair rather than unchanged reuse.

Partial witness repair is therefore useful for early improvement in dense packing, while broader reconstruction can produce a better solution with

more time. The experiments make this choice explicit in terms of size, density, and search budget.

Data and code availability

The benchmark generators, experimental results, and analysis and verification scripts are documented in the accompanying research materials. Identifying repository details are provided separately to the editorial office.

Supplementary material

The Online Supplement reports the primary 60-second comparison, including adaptive repair configuration, coverage, geometry-specific quality, target times, and timing checks (S1), followed by the one-second repair-mechanism study (S2). Secondary comparisons cover transfer, quality and diversity, amortized cost, reward and action configurations, completion checks, and constructor feasibility (S3 to S10). Feasibility, initialization, and selector comparisons are in S11; secondary methods and dataset coverage are in S12; policy architecture and training costs are in S13.

Declaration of competing interests

The authors declare no conflicts of interest.

Appendix A. Objective definition

Adjusted distance. Write c_i for facility i's center and $t = |\Delta y|/|\Delta x|$ for the slope of the segment joining the two centers. The directional radius r_i is the distance from c_i to the boundary of i's rectangle along that segment: the ray exits through a vertical edge when $t \leq h_i/w_i$, at offsets $(w_i/2,\, t\, w_i/2)$ from the center, and through a horizontal edge otherwise, at offsets $(h_i/2t,\, h_i/2)$; r_i is the Euclidean norm of the offsets. The adjusted distance is $d_{ij} = \|c_j - c_i\|_2 - r_i - r_j$, which is zero when the rectangles touch along the segment. Its normalizer $\bar{d}_{ij}$ is the same quantity evaluated with c_i at its lower left extreme $(w_i/2, h_i/2)$ and c_j at its upper right extreme $(W - w_j/2, H - h_j/2)$, floored at 10^{-12}.

Pair and boundary terms. With $\nu_{ij} = \min(d_{ij}/\bar{d}_{ij}, 1)^{\rho}$,

$$c_{ij} = \begin{cases} A_{ij}\,\nu_{ij} & A_{ij} \geq 0 \\ -A_{ij}\,(1-\nu_{ij}) & A_{ij} < 0, \end{cases}$$

zeroed if i and j overlap. The boundary term is

$$\begin{aligned} b_i \;&= w_{\mathrm{e}}\, h_i\, s_i\, \mathbf{1}[i \text{ flush left or right}] \\ &\quad + w_{\mathrm{e}}\, w_i\, p_i\, \mathbf{1}[\text{flush bottom}] + w_{\mathrm{e}}\, w_i\, q_i\, \mathbf{1}[\text{flush top}], \end{aligned}$$

where $(s_i, p_i, q_i) \in \{0,1\}^3$ is the wall preference archetype of i's type; b_i is zeroed whenever i overlaps anything.

Constants and the terms off $\mathcal{F}$. The constants are $w_{\mathrm{adj}} = w_{\mathrm{e}} = 2$ and $\rho = 1$. With $A(g)$ the total pairwise overlap area and $P = w_{\mathrm{ov}} A(g)$, the full evaluator is the pair sum plus the boundary sum plus $P + B(P)$. Here $w_{\mathrm{ov}} = -8$ and

$$B(P) = \begin{cases} \beta_{\max} = 400, & P = 0, \\ \beta_{\mathrm{mid}}(1 - P/\tau)^2, & \tau \leq P < 0, \\ 0, & P < \tau, \end{cases}$$

where $\beta_{\mathrm{mid}} = 200$ and $\tau = -200$. Therefore $P = 0$ and $B(P) = \beta_{\max}$ on $\mathcal{F}$, exactly as in Eq. (2); off $\mathcal{F}$, the overlap penalty and graded bonus apply.

Generated suite distributions. Each instance draws its mean facility area from $U(9, 30)$, capped at 0.95 of the budget for each facility, $G^2 \cdot \mathrm{fill}/n$; a plate aspect ratio from $U(1, 1.8)$; plate sides meeting the implied area, each at most G; $G = 44$ in the one-second study and $G = 44, 44, 72, 100$ for $n = 32, 64, 128, 256$, respectively, in the 60-second study; a guillotine partition with minimum side 2, shrunk to the target fill; a type count T uniform on $\{6, \ldots, \min(12, n)\}$; a symmetric integer affinity table on types, entries uniform on $\{-8, \ldots, 8\}$ and zeroed with probability 0.25; and a wall preference archetype per type. Three objective constants are drawn again once per suite cell, spanning the range the two reference scenarios occupy: $w_{\mathrm{ov}} \sim U(-12, -6)$, $\beta_{\max} \sim U(320, 520)$ and $\tau \sim U(-280, -160)$; all other constants are fixed at the values above.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex and Anthropic Claude for language editing, implementation assistance, and checking of numerical consistency. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Adya, S. N., & Markov, I. L. (2003). Fixed-outline floorplanning: Enabling hierarchical design. *IEEE Transactions on Very Large Scale Integration Systems*, *11*(6), 1120–1135. https://doi.org/10.1109/TVLSI.2003.817546

Bello, I., Pham, H., Le, Q. V., Norouzi, M., & Bengio, S. (2017). Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*.

Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, *290*(2), 405–421.

Bertsekas, D. P. (2005, April). *Rollout algorithms for constrained dynamic programming* (tech. rep. No. LIDS Report 2646). Massachusetts Institute of Technology, Laboratory for Information and Decision Systems. https://www.mit.edu/~dimitrib/Rollout_Constrained.pdf

Bertsekas, D. P. (2020). Constrained multiagent rollout and multidimensional assignment with the auction algorithm. *arXiv preprint arXiv:2002.07407*. https://doi.org/10.48550/arXiv.2002.07407

Burkard, R. E., Karisch, S. E., & Rendl, F. (1997). QAPLIB – a quadratic assignment problem library. *Journal of Global Optimization*, *10*(4), 391–403.

Chang, Y.-C., Chang, Y.-W., Wu, G.-M., & Wu, S.-W. (2000). B*-Trees: A new representation for non-slicing floorplans. *Proceedings of the 37th Design Automation Conference*, 458–463. https://doi.org/10.1109/DAC.2000.855354

Drira, A., Pierreval, H., & Hajri-Gabouj, S. (2007). Facility layout problems: A survey. *Annual Reviews in Control*, *31*(2), 255–267.

Fowler, R. J., Paterson, M. S., & Tanimoto, S. L. (1981). Optimal packing and covering in the plane are NP-complete. *Information Processing Letters*, *12*(3), 133–137. https://doi.org/10.1016/0020-0190(81)90111-3

Gasse, M., Chételat, D., Ferroni, N., Charlin, L., & Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. *Advances in Neural Information Processing Systems*, *32*, 15554–15566.

Glover, F. (1989). Tabu search—part i. *ORSA Journal on Computing*, *1*(3), 190–206.

Gomes de Alvarenga, A., Negreiros-Gomes, F. J., & Mestria, M. (2000). Metaheuristic methods for a class of the facility layout problem. *Journal of Intelligent Manufacturing*, *11*(4), 421–430.

Google OR-Tools. (2026). CP-SAT solver [Accessed 6 September 2026]. https://developers.google.com/optimization/cp/cp_solver

Haralick, R. M., & Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, *14*(3), 263–313.

Heinbach, B., Burggräf, P., & Wagner, J. (2023). Deep reinforcement learning for layout planning–an MDP-based approach for the facility layout problem. *Manufacturing Letters*, *38*, 40–43.

Ikeda, H., Nakagawa, H., & Tsuchiya, T. (2022). Towards automatic facility layout design using reinforcement learning. *FedCSIS (Communication Papers)*, 11–20.

Kakooee, R., & Dillenburger, B. (2024). Reimagining space layout design through deep reinforcement learning. *Journal of Computational Design and Engineering*, *11*(3), 43–55.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680.

Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! *International Conference on Learning Representations*.

Koopmans, T. C., & Beckmann, M. (1957). Assignment problems and the location of economic activities. *Econometrica*, *25*(1), 53–76.

Luo, G., Zhou, X., Feng, L., Liu, J., Liu, P., Liao, Y., Shan, W., & Qi, H. (2025). Controllable and flexible residential floor plan layout design based on multi-agent deep reinforcement learning with layout prior size and similar experience abandon. *Advanced Engineering Informatics*, *68*, 103702.

Mazyavkina, N., Sviridov, S., Ivanov, S., & Burnaev, E. (2021). Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, *134*, 105400.

Mirhoseini, A., Goldie, A., Yazgan, M., Jiang, J. W., Songhori, E., Wang, S., Lee, Y.-J., Johnson, E., Pathak, O., Nova, A., Pak, J., Tong, A., Srinivasa, K., Hang, W., Tuncer, E., Le, Q. V., Laudon, J., Ho, R., Carpenter, R., & Dean, J. (2021). A graph placement methodology for fast chip design. *Nature*, *594*(7862), 207–212. https://doi.org/10.1038/s41586-021-03544-w

Murata, H., Fujiyoshi, K., Nakatake, S., & Kajitani, Y. (1996). VLSI module placement based on rectangle-packing by the sequence-pair. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *15*(12), 1518–1524. https://doi.org/10.1109/43.552084

Ow, P. S., & Morton, T. E. (1988). Filtered beam search in scheduling. *International Journal of Production Research*, *26*(1), 35–62.

Pérez-Gosende, P., Mula, J., & Díaz-Madroñero, M. (2021). Facility layout planning. an extended literature review. *International Journal of Production Research*, *59*(12), 3777–3816.

Potvin, J.-Y., & Rousseau, J.-M. (1993). A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. *European Journal of Operational Research*, *66*(3), 331–340.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, *40*(4), 455–472. https://doi.org/10.1287/trsc.1050.0135

Şahin, R., Niroomand, S., Durmaz, E. D., & Molla-Alizadeh-Zavardehi, S. (2020). Mathematical formulation and hybrid meta-heuristic solution approaches for dynamic single row facility layout problem. *Annals of Operations Research*, *295*(1), 313–336.

Singh, S. P., & Sharma, R. R. (2006). A review of different approaches to the facility layout problems. *The International Journal of Advanced Manufacturing Technology*, *30*(5), 425–433.

Taillard, É. (1991). Robust taboo search for the quadratic assignment problem. *Parallel Computing*, *17*(4–5), 443–455.

Wu, Y., Song, W., Cao, Z., Zhang, J., & Lim, A. (2021). Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(9), 5057–5069.